Multimedia Tools and Applications (2025) 84:32841–32863
https://doi.org/10.1007/s11042-024-20506-2

ABRAGame: automatic bit rate adjustment for cloud gaming

Alejandra Armendariz[1] · **Jose Joskowicz**[1]

Received: 23 April 2024 / Revised: 26 November 2024 / Accepted: 6 December 2024 /
Published online: 23 December 2024
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract
Cloud Gaming has emerged as a promising alternative for enjoying high-quality video games on low-cost devices. These platforms run games on remote servers and stream multimedia content to the player's device over the network. While they offer numerous advantages, they also present challenges and issues due to their heavy reliance on network quality. This work focuses on the research and development of a real-time algorithm that performs early detection of potential visual degradations and automatically adjusts the server's encoding rate to prevent such issues. In doing so, it accomplishes the objective of maintaining a smooth gaming experience, even on congested networks, while concurrently enhancing the overall quality of experience of the users. The algorithm, called ABRAGame, was tested on fifteen games, covering four different genres, using a real Cloud Gaming platform owned by the company ABYA, which provides Cloud Gaming services in Latin America.

Keywords Multimedia transmission · Multimedia service · Cloud gaming · Adaptive bit rate · Real-time prediction · Quality of experience

1 Introduction

The world of video games on personal devices has become very popular within the consumer technology sector over the last 30 years [1]. Leading analyst agency Newzoo shows that the revenues of the virtual entertainment industry are steadily growing every year. The agency's 2023 report estimates that the industry's revenue as a whole was 188 billion dollars worldwide in 2023 [2].

On traditional systems, players have to install video games on their physical machines in order to play them. Cloud gaming (CG), on the other hand, uses a client-server architecture where the games are uploaded and processed in the cloud and the user sees the video of the game being played on a thin client [3]. CG provides easy access to games without the need for an expensive gaming computer [4].

✉ Jose Joskowicz
josej@fing.edu.uy

Alejandra Armendariz
aarmendariz@fing.edu.uy

[1] Universidad de la República, Montevideo, Uruguay

Audio and video transmission over the Internet is used in different types of systems, such as film, music or real time events streaming platforms. The variable network conditions between the servers and the end users demand the use of different strategies for automatic adjustments of encoding and bit rate, including the recent implementation of Mobile Edge Computing (MEC) techniques [5]. These types of systems can tolerate relatively long delays and can apply jitter buffers and retransmission techniques to ensure a good user experience even with networking issues. Furthermore, classical streaming systems can have different bit rates pre-encoded for the same material, and the system can decide which one to send to each user according to the available bandwidth. However, in CG, bidirectional interactivity is needed, and time requirements are critical. When a player executes a command in the game, that command must be transmitted over the Internet and go through all the logical and graphical processing on the server, encoding, packetization, transmission back and decoding before the user can see the action in the game [6]. All this processing must be done in a very short time interval. Depending on the type of game, delays as small as 100 ms can affect the playability. Buffering, transcoding, pre-encoded material at different bit rates and other classical techniques, cannot be applied to CG. While bit rate automatic adjustment in classical streaming appears to be a mature technology, the interactivity and strict timing requirements makes a CG system harder to optimize. Several automatic bit rate adaptation or edge computing techniques for the optimization of these kinds of systems are still under investigation [7, 8].

The purpose of this work is to design, develop and test an automatic bit rate adjustment algorithm for video games, which we have called ABRAGame, to prevent issues that may affect the playability. In order to perform this objective, an early prediction of potential issues in the network was designed. After a degradation has been predicted, the system automatically reduces the video codec bit rate in order to maintain a good gameplay at the expense of some reduction in video quality. We have investigated how much the codec bit rate should be reduced in order to eliminate video frame loss and minimize the impact on quality. Once an automatic bit rate reduction is performed after a frame loss prediction, the system also automatically increases the bandwidth of the codec once the network issues are resolved. The main contribution of this work is the development, evaluation, and implementation of an innovative algorithm that significantly enhances CG playability under network degradation conditions, common in mobile networks or home Wi-Fi settings. The algorithm is resource-efficient, runs on the client side, and requires no server-side modifications. Its performance was extensively tested on a real CG platform, showing very good results.

In a previous work [9], the authors introduced the prediction algorithm of the ABRAGame, which demonstrated promising initial outcomes. The current paper significantly builds upon this foundation by detailing the design and development of the bit rate adaptation component within the ABRAGame algorithm. Furthermore, this study elaborates on the implementation of the full algorithm and conducts a comprehensive evaluation on an actual CG platform. While the prior work limited its scope to just four network scenarios, the present research broadens this by integrating additional scenarios. Moreover, it assesses the performance of both the prediction mechanism and bit rate adjustment across different network delays. This enhancement allows for an examination of conditions affecting users who are located thousands of kilometers away from the data center, thus offering a more robust and extensive evaluation of the algorithm's efficacy in diverse operational environments.

In the following sections, we will show how the designed algorithm works. Section 2 describes the main CG platforms at this moment, and present ABYA Go [10], the CG platform where all the testing was performed. Then various concepts of CG are presented. In Section 3, we present related works in which this work was based. Section 4 shows our experimental

setup and the different scenarios in which different types of video games have been tested. Section 5 describes the system architecture. Sections 6 and 7 are destined to explain how the designed algorithm works. Finally, we present the results in Section 8, along with Future Work in Section 9 and Conclusions in Section 10.

2 Cloud gaming systems

2.1 Cloud gaming service providers

In the late 2000s, cloud computing services for video games emerged, such as OnLive and Gaikai [3]. Gaikai was acquired by Sony, a major console manufacturer, which was a milestone in the industry. Currently, two prominent platforms are Nvidia's GeForce Now [11] and Microsoft's Xbox Cloud Gaming [12]. Both platforms reflect the gaming industry's trend toward using cloud computing to offer more accessible and flexible gaming experiences. These two companies are available primarily in the United States and Europe, and in other regions through partners. The rest of the offering is made up of emerging platforms, such as Amazon's Luna (2020, only available in the United States), and others that have been around for a while, but have gone through several changes, such as Google Stadia or Sony's PlayStation Now, now rebranded as PlayStation Plus. However, none of these platforms is available for Latin America [13].

The CG platform ABYA Go [10], developed by the company ABYA founded in Uruguay in 2015, offers this service in Latin America. In 2021, ABYA signed an alliance with Nvidia, promoter of GeForce Now, to be partner and offer their games in the region [14]. This work was developed using games from the ABYA Go platform. ABYA Go is made for a casual user. It works as a "Netflix for video games", meaning that the subscriber has access to a catalog of games already preloaded and all free-to-play. ABYA provided the team with a space on the *staging* server that is used for internal company testing. In addition, a data logging system was developed, which will be described in more detail in Section 5.

2.2 Cloud gaming typical architecture

A general outline of a CG architecture is shown in Fig. 1. Player commands are sent over the Internet from the client to the game server. Once the commands reach the server, they are converted into in-game actions and processed by the game logic module, which translates the changes to the game world. The changes in the game environment are processed by the system's graphics processing unit (GPU) and translated into a rendered scene. Subsequently, the rendered scene must be compressed by the audio and video encoder, and then sent to a streaming module, which delivers the multimedia stream to the client. Finally, the client decodes the audio and video, and displays the resulting frames and audio on the player's device [3, 15]. Stream transmission is typically carried out using Real-Time Protocol (RTP) or Secure Real-Time Protocol (SRTP). Particularly, the ABYA Go platform employs a customized version of RTP that operates over standard User Datagram Protocol (UDP).

2.3 Interaction delay

As mentioned in the previous section, a CG system must go through multiple processing stages after capturing the player's actions. To achieve good interactivity, all these operations

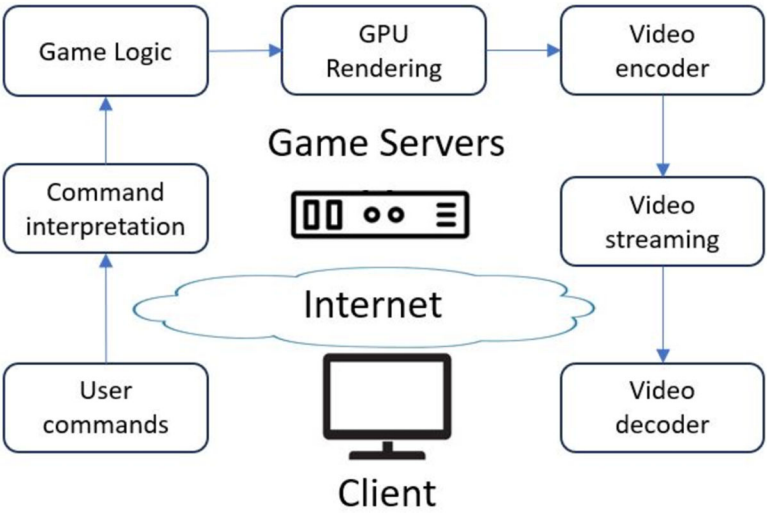


Fig. 1 Cloud Gaming typical architecture modules

must be performed in a matter of milliseconds. This time interval is defined as interaction delay [15, 16]. Different types of games have different delay tolerances. First-person shooter games (FPS) such as "Counter Strike" are very sensitive and can be affected with only about 100 ms of delay. Role-playing games (RPG) can withstand up to 500 ms of lag without players noticing it severely because the actions are performed by an in-game character. Finally, real-time strategy (RTS) games can tolerate up to 1 s of delay, as the actions tend to have a longer duration [7, 15]. Even when some type of games can tolerate delays of hundreds of milliseconds, CG system's design must be performed to work with the more restrictive type of games (i.e., FPS, where 100 ms delay affects the playability).

2.4 Video encoding in cloud gaming

H.264 [17] is one of the most commonly used codecs in CG systems because it can adapt well to real-time requirements [15]. The ABYA's system implements H.264 with an infinite Group of Pictures (GOP) without any B frames, as recommended by [18, 19] for CG. This means that, at the beginning of the transmission, the server sends a single I frame, followed by P frames, and ideally, this sequence would be maintained until the end of the session. In reality, the delay and packet loss present in the network can affect negatively in the decoding of P frames. If a packet containing information of a P frame is not available at the time of decoding, either because it arrived late at the decoder or because it was effectively lost, visible degradations occur in the current and subsequent decoded frames. This can result in frozen screens and/or undesired artifacts in the video. In this context, it becomes necessary to retransmit the reference I frame, which implies a higher consumption of bandwidth. Furthermore, this retransmission should occur instantaneously after detecting the loss, which means that this bigger bandwidth consumption (i.e., sending an I frame instead of a P frame) may occur before the channel condition improves, further worsening the situation. This highlights the importance of preemptively detecting and preventing losses in the client system.

2.5 Cloud Gaming bandwidth patterns

Video games have different states, that shows different characteristics in both gameplay and bandwidth needs [21]:

- **Loading screen:** A screen with some text that appears while the server is preparing the game.
- **Main Menu:** A screen with text items (e.g., new game, continue, options, etc.), and a background that can have multimedia content.
- **Play:** The player interacts and plays the game.
- **Idle:** Inside the game, but with the player not performing any actions.
- **Pause:** A text menu, typically overlapped on top of the frozen playing screen.

Figure 2 shows an example of the bandwidth pattern for the different states of the game Classic Racers. This bandwidth pattern was measured in a non-congested network with broadband Internet access. While the Idle and Pause states have almost constant bit rates, the Play state exhibits more variations in response to the different scenes produced during gameplay. Any degradation in the network that constrains the available bandwidth below the required level will affect playability.

3 Related works

A network analysis on some CG platforms (Stadia, GeForce Now and PlayStation Now) was performed by [20]. This study shows that, depending on the network and video resolution, the needed bandwidth for smooth playing can vary from 13 Mbps (for 720p) up to 45 Mbps (for 2160p or 4K). It also shows that these three platforms have built-in functionalities to test the network between the client and the server, in order to determine if the conditions are sufficient for a stable gaming session. Nevertheless, the mechanism used for bit rate adaptation in each platform is not described.

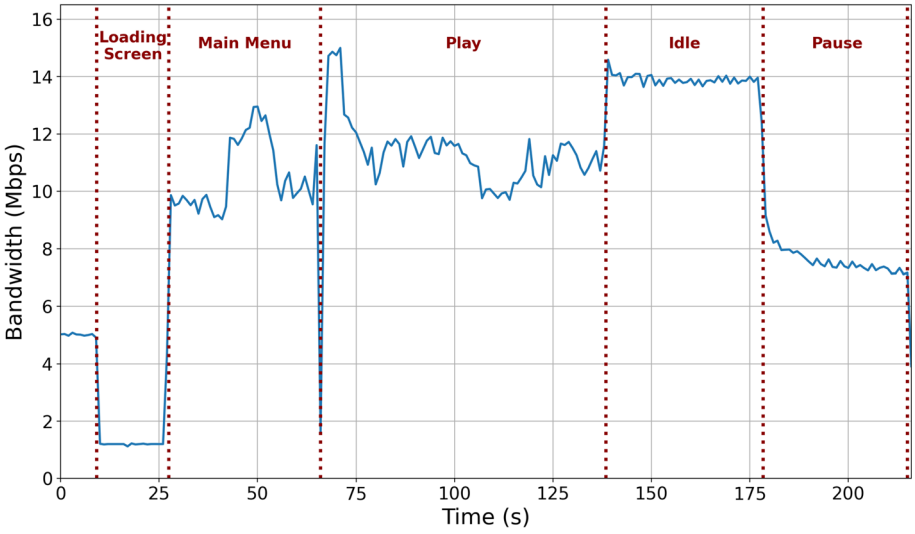


Fig. 2 Example of bandwidth consumption in various states of a typical Cloud Game

An extensive analysis of Stadia traffic patterns and behavior was conducted by [21]. This study shows the ability of Stadia to adapt to different link capacity conditions, including cases where the bandwidth drops or increases suddenly. According to [21], Stadia swiftly responds to reductions in available link capacity by entering an adjustment phase, which can persist for more than 200 seconds. During this phase, Stadia strives to establish a new configuration that offsets the diminished network resources. Throughout this period, the quality of the experience is significantly compromised. This includes frequent alterations in resolution, erratic frame rates, and occasional audio stuttering.

In the study [22], empirical measurements conducted on different CG platforms (Stadia, GeForce Now, xCloud, and PlayStation Now) are detailed. These assessments were carried out under various network constraints, including packet loss, bandwidth reduction, latency augmentation, and jitter fluctuation. The results demonstrate that these four platforms exhibit distinct adaptation behaviors, frequently leading to a deteriorated quality of user experience. According to this study, the application-level mechanisms employed in these platforms tend to exhibit prolonged detection and response times to network irregularities, rendering the service marginally playable and resulting in significant stuttering from the user's perspective.

The performance of CG on wired and wireless networks have been recently studied by [23] and [24]. Both studies show that clearly, wireless environments (such as Wi-Fi or mobile cellular networks) are more unstable than wired LANs, and CG services can suffer from these network instabilities, such as sudden drops or changes of bandwidth. [25] analyzed the performance of cloud gaming over Starlink, a Low-Earth Orbit (LEO) satellite network. This type of network has higher and faster changing latency, less stable bandwidth, and more packet loss than other wireless networks, putting additional challenges to the playability of CG.

The idea that there is a direct relationship between the delay experienced by packets from the time they are sent by the server until they are received by the client, and the increase in the network buffer queue, as well as the fact that the overflow of the network buffer leads to packet losses, has been studied and used by many authors to predict packet losses and estimate the bit rate of transmission. In [26] the Google Congestion Control (GCC) is presented. This algorithm infers congestion based on the queuing delay gradient from a Kalman filter, and is used in WebRTC video conferencing applications. It has been tested with bandwidths up to 3 Mbps and queuing delays up to 700 ms, compatible with videoconferencing, but not with CG.

The authors of [27] presented a congestion control mechanism, called C3G, designed for CG, and proposes to detect the instant when the network buffer starts to fill by comparing the bits sent by the server $\lambda(t)$ with the bits received by the client $\mu(t)$. This is done during equal time intervals throughout the game. If in any of these measurement intervals $\lambda(t) > \mu(t)$, it is interpreted that there are packets accumulating in the buffer. From the instant t_0, the Round Trip-time Video Latency (RTLV) is measured. If it exceeds a certain threshold, the server is told to lower the sending bit rate to prevent the buffer from continuing to fill up. This causes the buffer to start emptying and consequently reduces latency.

As presented in the previous paragraphs, CG platforms have mechanisms to make automatic bit rate adjustments of their media based on the network congestion. Nevertheless, these mechanisms do not perform well in all situations, especially when accessed through wireless networks, leaving space for the design of novel and better algorithms to perform this task.

Taking the queuing delay gradient to infer network congestion, as in [26], produces a significant number of false predictions of visual degradation in cloud gaming, as will be described in Section 6. On the other hand, the method proposed by [27] to measure the delay (or buffer size), requires developing part of the algorithm directly on the server, the permanent transmission of data from the server to the client, and a specific synchronization mechanism.

This may cause extra load to the server and the network, which both are undesired in these kinds of systems.

This work is based on the aforementioned background, but improving and avoiding its weaknesses. As described in the following sections, delay is used as one of the main predictors of impairments, while avoiding the false prediction rate caused by the use of the gradient. On the other hand, the entire system is developed and can run on the client side, avoiding any additional development or load on the server.

4 Test conditions and experimental setup

In this work, we adopted the widely used network model where a complex IP network path is simplified to a single link, known as the bottleneck link [28]. This link's bandwidth, the lowest among all traversed links, defines the network's transmission speed. Packet latency in the test IP network includes the overall network delay.

Test conditions were designed in order to emulate different real scenarios. Most cloud gaming users use home Wi-Fi networks for their connection to the cloud server. These users are located in different countries, and may be thousands of kilometers away from the cloud Data Center. Internet typical delay may vary between few milliseconds for users located near the Data Center and up to 50 ms for users located at distant cities.

The initial tests were conducted using a typical home Wi-Fi network to determine the most common degradations perceived by users in such networks, as shown in Fig. 3. These tests were conducted under various conditions. They included playing games at different proximities to the Wi-Fi access point, while simultaneously running or initiating video streaming applications (such as YouTube or Netflix), with multiple players sharing the same Wi-Fi network concurrently, alongside other common situations. Figure 4 presents examples of bandwidth fluctuations observed during the tests. These graphs were generated by measuring internet bandwidth over a 240-second period under various conditions. A custom script was used for the measurements, running a "speed test" in a continuous loop throughout the testing period.

Based on the results obtained, a controlled test bed was developed and deployed, as shown in Fig. 5. It consists of a client computer running the CG application simultaneously with the algorithm developed in this work. It is connected to the Internet through another computer configured as a Layer 2 bridge, which runs a network emulator. The connections are made using network cables, including the connection from the bridge to the router for Internet access.



Fig. 3 Examples of typical degradation when frames are lost. In the left image, a normal example of the game V-Rally 4. In the right image, a degraded example of the same game

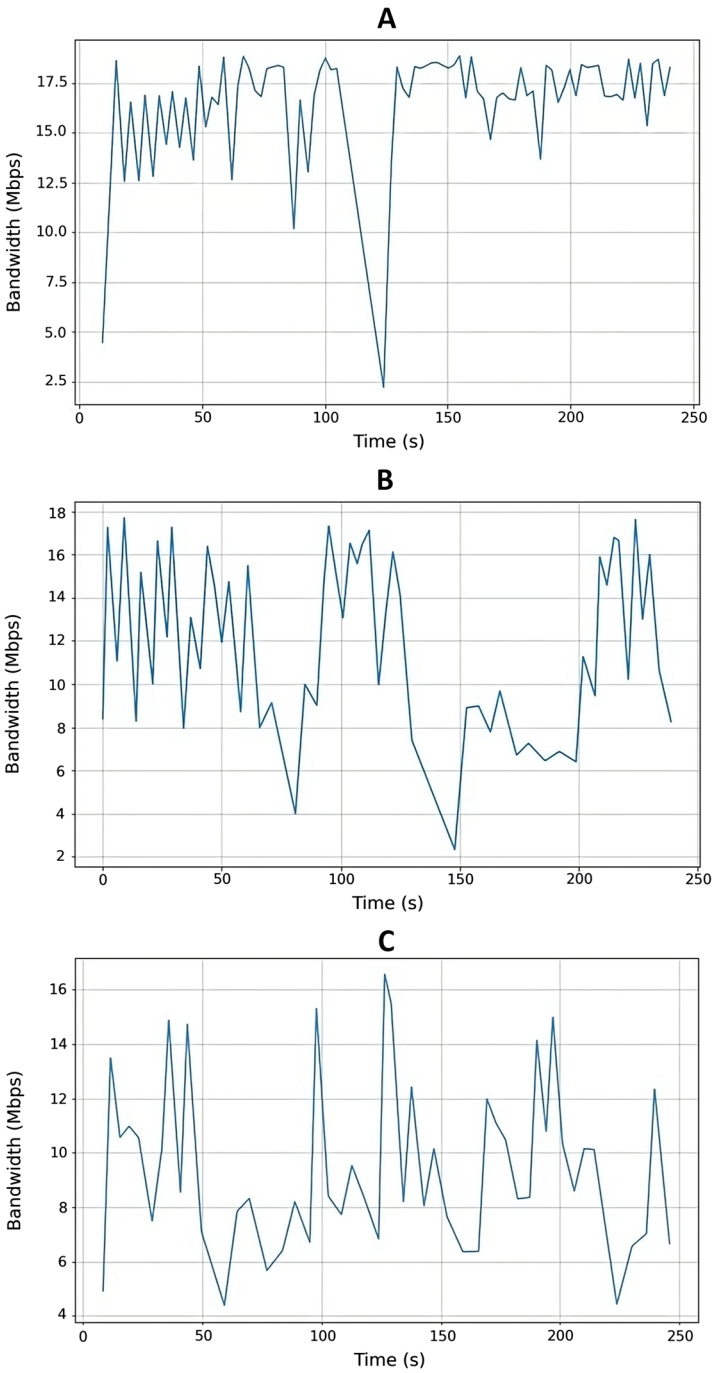


Fig. 4 Examples of three typical Wi-Fi bandwidth patterns, obtained from a home network

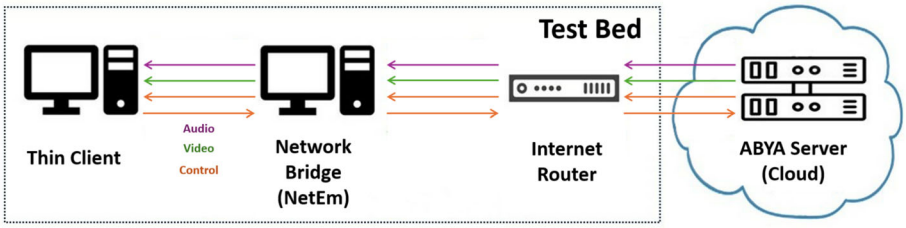


Fig. 5 Test bed connection diagram

To implement the network bridge, we used a computer with Ubuntu 20.04 and two local area network interfaces (LAN). The brctl command, which is part of the bridge-utils package, was used to connect the adjacent network segments to both interfaces, enabling transparent IP packet exchange between the client and the CG server located in the cloud.

Network conditions emulation is performed using the NetEm application [29], which also runs on the bridge. Controlling the channel bandwidth, the bottleneck link can be emulated. Controlling the delay, the situation of distant users can be considered. Six representative situations encountered in home Wi-Fi contexts were used to evaluate the impact on different games, such as the ones present in Fig. 4.

Based on this, we developed various test configurations illustrated in Fig. 6. Each configuration was executed for 240 seconds, during which the network bandwidth varies to emulate different network conditions. Figure 4-A illustrates a sharp decline in bit rate followed by an equally rapid recovery, mimicking a network 'glitch' corresponding to condition A. Scenarios B and C demonstrate two patterns of gradual degradation with swift restorations, similar to those shown in different moments of Fig. 4-B. Scenario D is characterized by intermittent fluctuations, exhibiting both increases and decreases in an unstable environment, as depicted

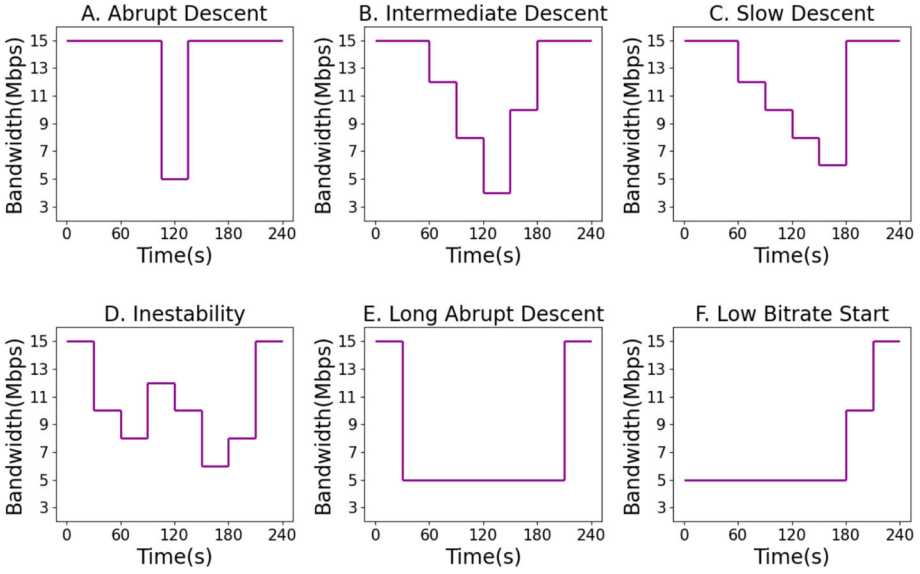


Fig. 6 Test settings of bandwidth limitation implemented with NetEm

in Fig. 4-C. Scenario E simulates a sudden drop in network efficiency, which is followed by an abrupt recovery after a brief period. Finally, Scenario F begins with a congested network operating at a very low bit rate, which then experiences a rapid improvement towards the end of the evaluation period.

These network conditions were implemented in two cases: one where the client is near the server (with no network delay) and another where the user is located thousands of kilometers away from the data center. In the latter case, an additional 50 ms delay, corresponding to the average network latency under these conditions, was introduced. To streamline the implementation of emulation conditions, a script was developed to automate changes in NetEm based on each test configuration.

5 System architecture

One of the main challenges of this work was to access, in real-time, a set of variables of interest generated by the ABYA server and modify the video encoding rate during gameplay. This process carries the risk of affecting the company's service provided to its customers. Therefore, it was necessary to design a non-invasive solution that would allow us to continue the research without jeopardizing the streaming service. For this purpose, the ABYA team provided a version of the client application that includes a real-time logging system and the ability to modify the codec's encoding rate remotely, from the client side, without the need to access the server at any time. This allowed to carry out the research effectively and safely, without risking the streaming service.

While a game is being played, the measurements of a number of chosen variables are logged in real-time on the client side. ABYA's platform uses 60 fps for the video stream, so a frame is received every 16.7 ms. The logs are updated every 20 ms, and the different network parameters are calculated for each 20 ms window. The main variables used in the development of the algorithm are defined as follows:

- *Average Network Latency (AvgNetLat):* This variable measures the delay that packets experience from the time they are sent from the server until they are received by the client. It can be calculated by dividing the total delay of all the received packets by the number of packets received during the measurement interval, as shown in (1):

$$AvgNetLat = \frac{\sum(T_a - T_d)}{Packets\ Received} \tag{1}$$

  where T_a and T_d represent the time when a packet is received and sent, respectively. Note that client and server must be synchronized, but this is a standard feature implemented between ABYA's client and server. The calculations are performed at the client side, so there is no need for any overload in the server for this metric.

- *Estimated bandwidth (estBW)*: This metric can be calculated as the sum of all received bits divided by the duration of the measurement window, as shown in (2). It is important to note that this estimation does not reflect the actual capacity of the communication channel, but rather the bandwidth of the audio and video streams received by the client.

$$estBW = \frac{\sum Bits\ received}{Window\ duration} \tag{2}$$

  This metric is also calculated at the client side.

- *Average bit rate (Avgbit rate)*: This parameter indicates the encoding rate at which the video codec operates on the server. It can take values higher than 1 Mbps and can be modified during a gameplay. This metric is known at server side, and transmitted to the client.

Figure 7 shows a general scheme of the system. On the right side, the ABYA company servers, located at the Data Center. On the left side, the user's device (i.e., a PC). Between them (not shown in the figure), the internet and the network bridge, as explained in Section 4.

The user's device runs the ABYA client application, enabling access to the game catalog and the ability to play the chosen game. Upon game selection, the ABYA client application generates a log file that periodically writes network variable values. Simultaneously, the designed algorithm is executed within a program on the same device. A high level block diagram of the program is presented in Fig. 8. This program seamlessly accesses the real-time log file and interprets the network variables produced by the ABYA client application. The program awaits the start of a new game, initializes and triggers the "Prediction of quality degradation" algorithm. Should the algorithm predict potential visual degradation or network improvements, it triggers the "bit rate adjustment" function. This function calculates the new encoding bit rate value and sends it to the ABYA client application. Consequently, the ABYA client application communicates with the server to modify the encoding bit rate accordingly. Throughout the game, the "Prediction" algorithm persistently monitors network parameters. Whenever needed, the algorithm adjusts the bit rate, employing the "bit rate adjustment" function again. Upon conclusion of the game, the program proceeds to the finalization phase, awaiting the initiation of a new gaming session. The detailed design of the "Prediction of quality degradation" algorithm and the "bit rate adjustment" function are explained in the next sections. With this architecture, all the needed processing is performed at the client side, without the need to modify the server or even the ABYA client application.

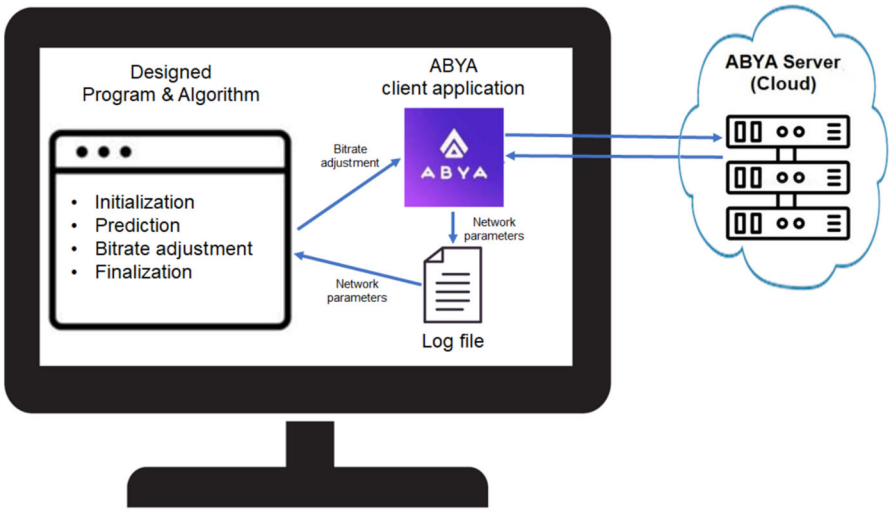


Fig. 7 A schematic description of the designed system

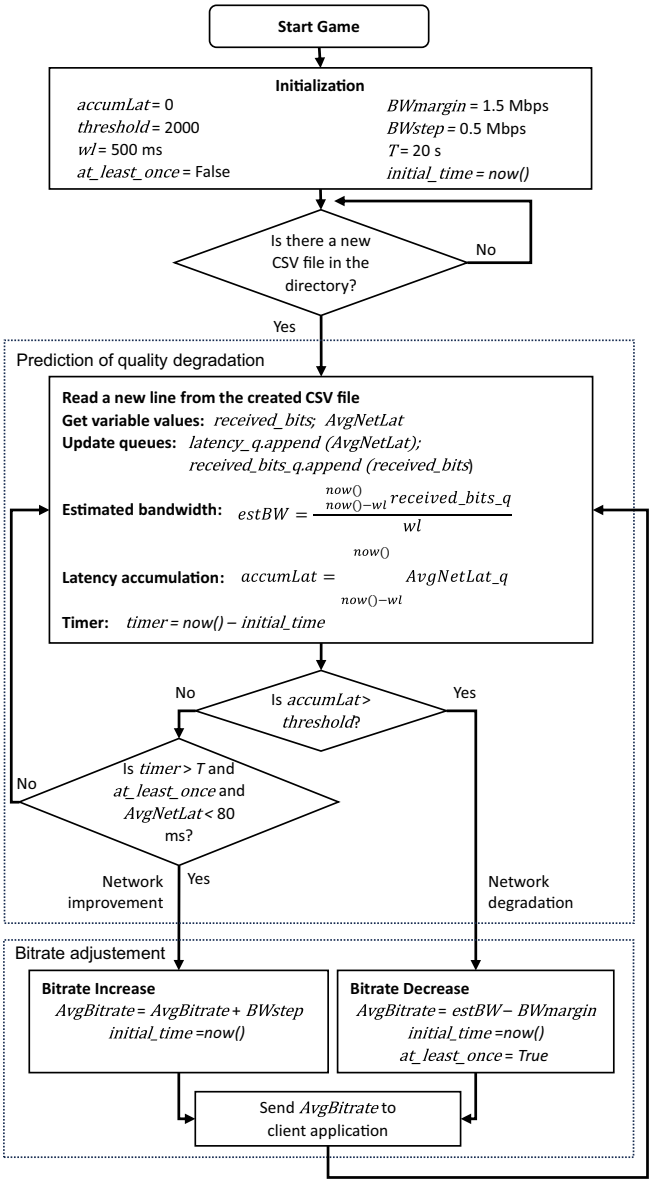


Fig. 8 Block diagram of the designed program and algorithms

6 Prediction of video quality degradation

Video degradations occur when a video frame is lost. By analyzing the obtained logs and various network metrics, it was possible to establish a strong correlation between frame loss and network latency, observing that latency increases sharply just before the loss occurs in all cases. An example of this behavior can be seen in Fig. 9.

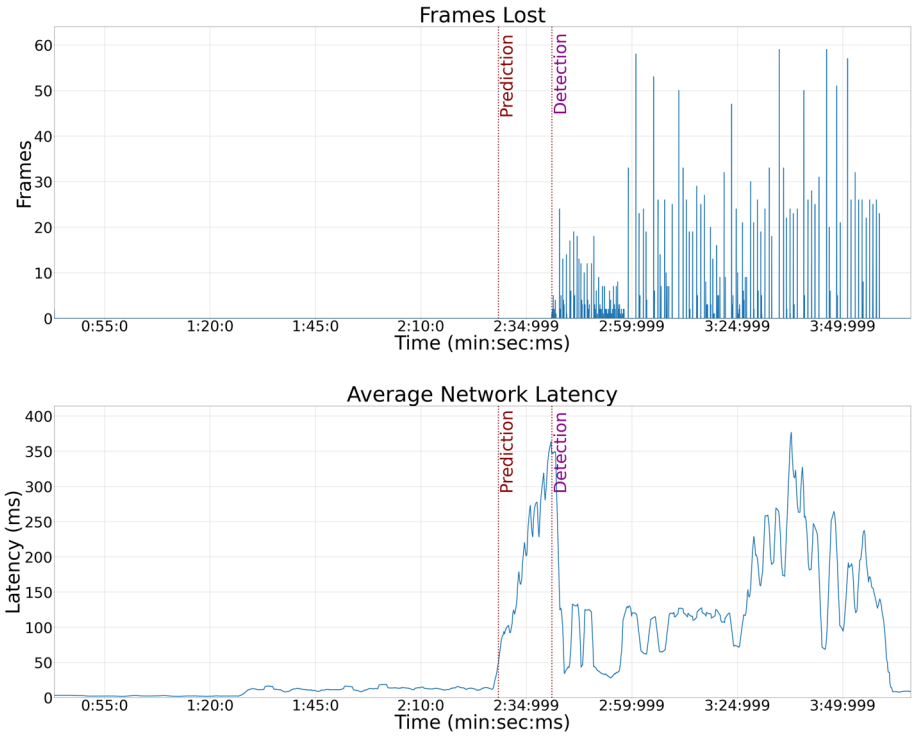


Fig. 9 A typical case characterized by a sudden increment of network latency, followed by the consecutive loss of multiple frames. The red line indicates the moment our algorithm predicts the loss, while the purple line identifies the onset of actual degradation

In the initial phase of our work, we developed a preliminary algorithm that uses the latency slope to predict degradation, similar to the C3G algorithm. The implementation uses a 200 ms moving windows to calculate the slope that best approximated latency. If this value exceeded a certain threshold, a degradation alarm is triggered. Several values for the window sizes and thresholds were tested during this phase to optimize performance. The algorithm was tested on different games genres in 183 game runs (more details are given in next sections). The results of this algorithm, summarized in Table 1, were not promising. This led us to aim for a new algorithm, that could significantly reduce false positives and more accurately predict all instances of degradation.

As a result, we developed a new "Prediction of quality degradation" algorithm, to predict degradations in real-time, based on the cumulative latency instead of the slope [9]. During

Table 1 Summary of prediction results for an algorithm based solely on the slope of the latency

	Launched games	Total predictions	Success predictions	Unpredicted degradations	False Positives
TOTAL	183	545	240	29	305
%		100%	44%	11%	56%

gameplay, as explained in the previous section, this algorithm runs in parallel on the computer of the client.

The system uses moving windows to sum latency values, which are measured every 20 ms, as described in Section 5. The duration of the moving windows and the cumulative latency threshold were established through extensive preliminary testing. These tests aimed to determine optimal parameter values by examining the cumulative latency over different window lengths, in scenarios with and without frame loss. It can be seen that longer window lengths are more sensitive to small variations in latency that may not result in frame loss, while shorter window lengths reduce the response time available for adjustments. A comparison of different window lengths is shown in Fig. 10. The performance of various threshold values was evaluated at different moving window lengths, and finally the most effective empirical values were selected to integrate into the algorithm. This resulted in 500 ms for the moving window length (*wl*) and a value of 2000 for the accumulated latency in this window, measured every 20 ms (as an example, a constant latency slightly above 80 ms for 500 ms will trigger the threshold). When the program detects that the accumulated latency exceeds the threshold, the algorithm triggers a degradation alarm, indicating an imminent degradation.

As we mentioned earlier in Section 3, we decided to implement our predictor program based on latency accumulation instead of its slope increment in order to reduce the number of false positives. In Fig. 11, we can observe a scenario in which, even though the latency slope increases abruptly, no frame loss is detected in the subsequent moments. Our implemented algorithm does not react to the detected increment in this case, preventing an eventual unnecessary reduction of codec bit rate.

7 Automatic bit rate adjustment

7.1 Bit rate decrease

The main objective of this work is to achieve an automatic adjustment in the coding rate of the video codec, in order to avoid visual degradations. Once a loss prediction is made, the designed algorithm is set to automatically adjust the video codec's encoding rate to prevent it. In this context, the channel is considered to be nearing a state of bufferbloat [27, 28]. Since it's not feasible to determine the channel's capacity non-intrusively at all times, we opt to utilize the variable *estBW* as defined in Section 5. This variable measures the video bit rate received by the client at each moment. Based on this, we have decided to configure the server encoding bit rate a margin lower than what is being received, aiming to prevent complete channel saturation. For this task, we've implemented a function that calculates the optimal bit rate for the server to transmit. This function aims to find the maximum encoding bit rate that ensures good gameplay while utilizing the available channel conditions effectively. As mentioned earlier, the *estBW* variable serves as a reference point, and the codec is instructed to reduce the encoding bit rate by a *BWmargin* of 1.5 Mbps below the *estBW* value obtained at the moment of the degradation prediction. This empirical margin was established after several tests, where it was observed that a smaller margin didn't successfully prevent visual degradation, and a larger margin maintained playability at the cost of game resolution. The adjustment to the new bit rate occurs in discrete steps (*BWstep*) of 0.5 Mbps.

In Fig. 12 it can be seen that, after a prediction, the algorithm automatically reduces the codec bit rate, thus avoiding degradations. It is noteworthy that here there are no more

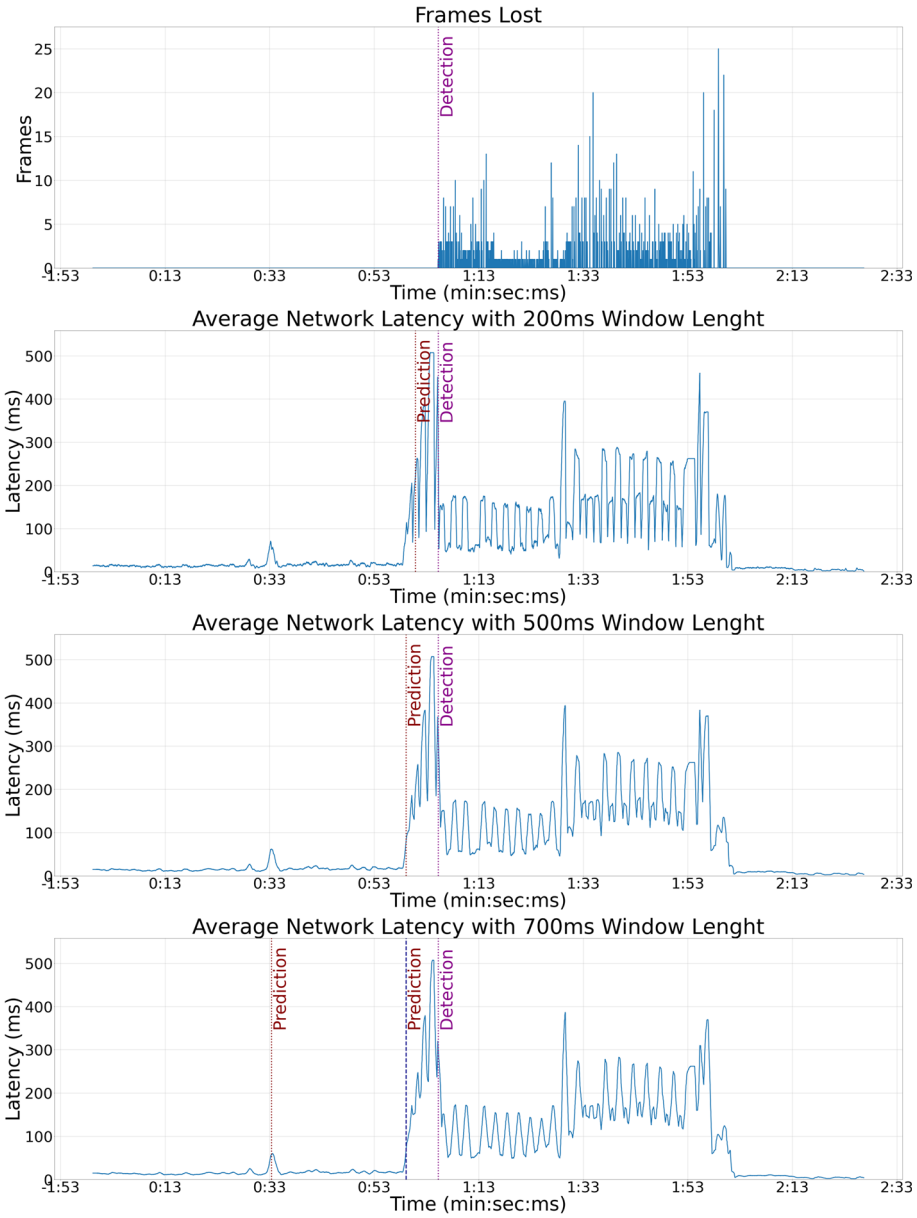


Fig. 10 A comparison of different window lengths of 200ms, 500ms and 700ms

dropped frames, despite the network conditions. Also, it can be seen that the latency returns to its initial state, which maintains the playability in the game.

Two key considerations that must be taken into account are that the bitrate adjustment control signal effectively reaches the server and that the bitrate adjustment is performed before any potential loss occurs. While a downlink channel may experience congestion at the moment of predicting a loss, this is not typically the case for the uplink channel, which generally opates

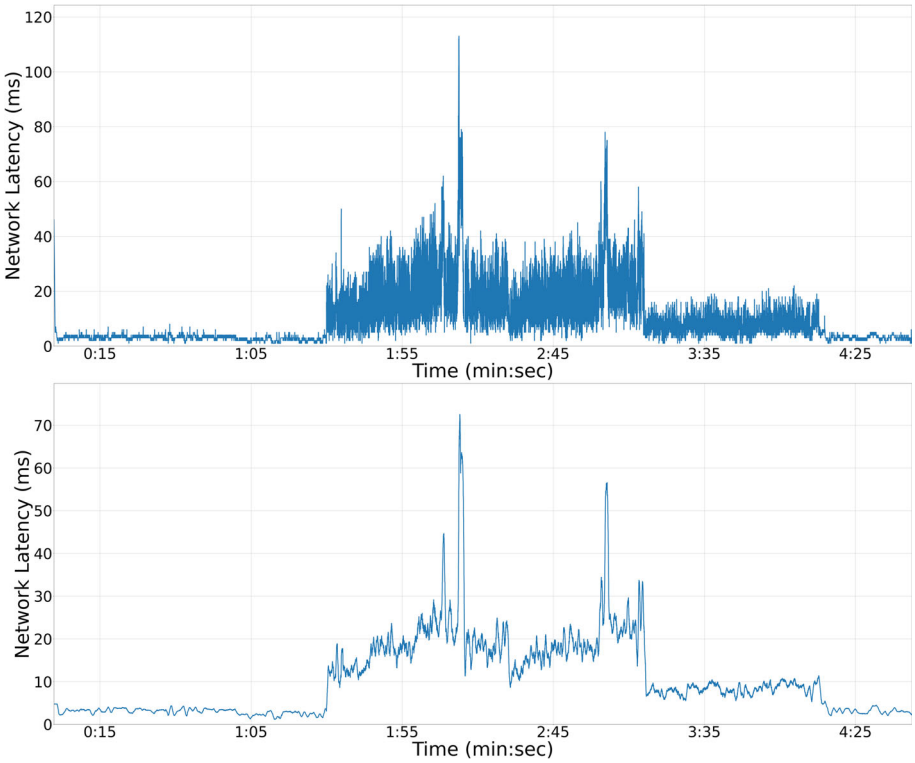


Fig. 11 A gameplay scenario where a sudden increase in network latency occurs, with a very high slope value, but no loss of consecutive frames is detected. The upper image shows the instantaneous values obtained from the network latency, while the lower shows the latency accumulation over a 500 ms window. In this case, the implemented algorithm does not trigger a degradation alarm, as we are below the threshold value, thus preventing an unnecessary reduction in resolution

under normal bandwidth conditions. The bitrate adjustment signal is sent through the client's control channel with reception confirmation, guaranteeing delivery to the server. The total response time for the entire process-from prediction to the actual bitrate adjustment on the client side, including the necessary processing on the server-has been approximately 500 ms. This response time has consistently been sufficient to prevent degradation, as evidenced in the example shown in Fig. 12.

7.2 Bit rate increase

Figure 13 shows a typical scenario of the algorithm behavior during the execution of a game. The moment a prediction occurs, a timer is started. If no prediction occurs within 20 seconds, the encoding rate value is increased by 0.5 Mbps and the timer is restarted. Conversely, if a new prediction occurs during that time, the bit rate reduction algorithm explained above is implemented. The value of 20 seconds was chosen to have a waiting margin in case of possible degradation predictions, and not to have to lower the bit rate even more abruptly after an upload. Note that the drop in video bit rate around 2:30:00 corresponds to a static screen that does not require as much bandwidth, such as a game menu or a loading screen.

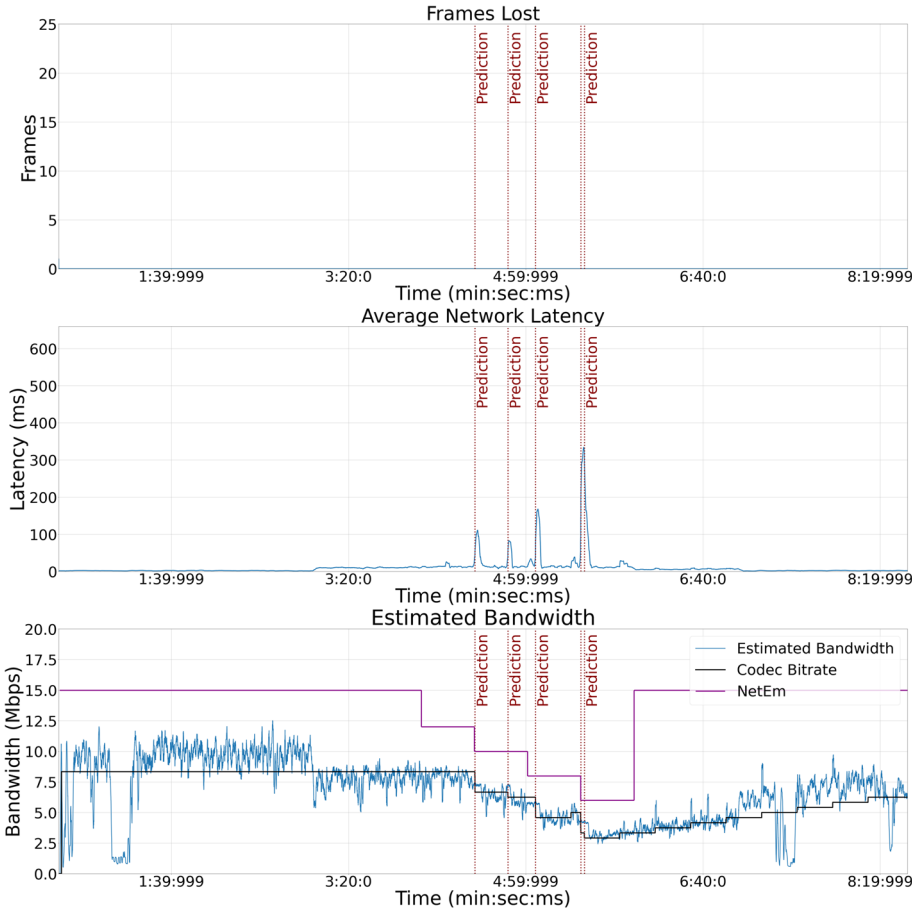


Fig. 12 Top: frames lost. Center: average latency. Bottom: Channel bandwidth (emulated with NetEm), encoding bit rate (as sent by the server) and estimated bandwidth received by the client. After a loss prediction is made, the algorithm automatically reduces the transmission encoding bit rate, and it is shown that frame loss is cancelled, thus avoiding visual degradations

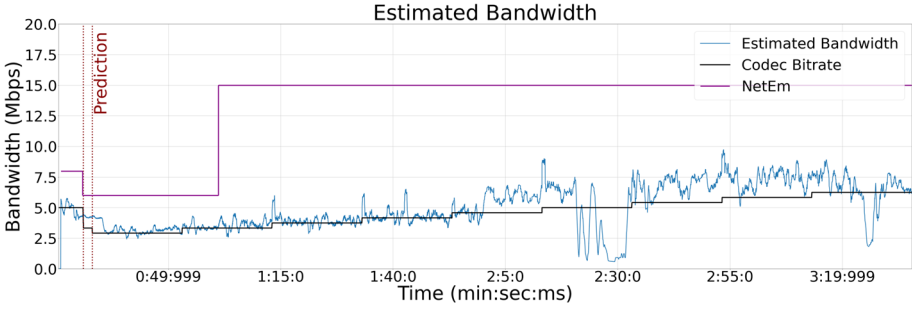


Fig. 13 A gameplay showing how the bit rate increase of our algorithm works. If no prediction occurs within 20 seconds, the encoding rate value is increased in steps of 0.5 Mbps

A stepwise linear incremental strategy was used for the implementation of the bit rate increase. This decision was made due to the lack of channel bandwidth information, which makes it difficult to develop an estimator to determine system bandwidth availability.

8 Results

8.1 Prediction of video quality degradation and automatic bit rate adjustment

The predictive algorithm was tested on 15 video games, covering 4 different genres, with a total of 183 games played. Table 2 lists all games used, with their respective genres, brief descriptions, and game title abbreviations. The first four network scenarios presented in Fig. 6 were implemented for the development of the prediction algorithm. Fast-paced games were selected, as they are more demanding in terms of network latency. In this stage of the testing process, it is also assumed that both the server and client were close, resulting in a base latency no greater than 7 ms. The implemented conditions and the results obtained for each game can be seen in Table 3.

Table 4 presents a summary of the results obtained in the prediction algorithm, segmented by genre. The algorithm made a total of 300 predictions out of 269 observed visual degradations due to frame loses. There were no unexpected degradations, although a false positive prediction rate of 10.3% was obtained.

It is important to note that all visual degradations were successfully predicted. In this application, it is preferable to have a few false positives rather than missing any real degradation events, as this would result in a decrease in the QoE of the user [30].

Table 2 List of tested games

Game Title	Genre	Brief description	Abbr.
Alien Rage	FPS	FPS game set in space with aliens	AR
Cyber Hook	FPS	First person 3D parkour skills game	CH
Classic Racers	Racing	Racing game with classical cars	CR
Extinction	RPG	Action RPG sword fighting game	EXT
Gravel	Racing	Extreme racing game	GRA
Grip	Racing	Futuristic combat racing game	GRI
King of Fighters XIII	Fighting	Fighting game in The King of Fighters series	KOF
Lord of the Fallen	RPG	Action RPG game set in a medieval fantasy world	LOTF
MXGP 2020	Racing	Official Motocross World Championship game	MXGP
Outcast: Second Contact	RPG	RPG action adventure set in an alien world	OSC
Stay Safe	Racing	Racing game with obstacles die and retry style	SS
Styx: Master of Shadows	RPG	RPG game set in a dark fantasy universe	STYX
Tandem	RPG	Puzzle platformer game	TAN
V-Rally 4	Racing	Realistic rally racing game	VR4
WRC 8	Racing	Official World Rally Championship game	WRC

Table 3 Results of the prediction of video degradation

Title	Launched games	Total visual degradations	Predictions	Tested network conditions
AR	7	11	12	A - D
CH	20	32	37	A - B - C - D
CR	20	37	39	A - B - C - D
EXT	7	8	11	A - B
GRA	7	7	8	A - B
GRI	20	29	32	A - B - C - D
KOF	20	39	43	A - B - C - D
LOTF	20	25	25	A - B - C - D
MXGP	7	10	10	A - D
OSC	7	14	18	B - D
SS	7	8	9	A - B
STYX	7	10	13	A - C
TAN	7	17	20	A - B
VR4	7	7	7	A - B
WRC	20	15	16	A - B - C - D
TOTAL	183	269	300	

The automatic bit rate adjustment algorithm was tested on 9 games out of the 15 previously used for degradation prediction, for a total of 182 games played, with two different latency values (7 ms for “low” and 50 ms for “high” latency). When a game is played without the algorithm, gameplay is severely affected, or even the game is completely lost and not recovered when conditions become favorable again. The algorithm maintains the ability to continue playing despite poor network performance, latency and low bandwidth, according to our tests. Table 5 shows the detailed results.

Finally, Table 6 provides a summary of the overall results obtained, showing an effective cancellation rate of 85.9%. It is important to highlight that in all instances, gameplay is greatly improved, even in cases where the loss could not be effectively avoided. In these situations, the interruption duration was reduced to an average of 4.5 s, whereas originally, it exceeded 20 s or, in some cases, the game could never be resumed. During informal testing, various players were presented with the same game under network degradation conditions, both

Table 4 Summary of prediction results

Genre	Launched games	Total predictions	Success predictions	Unpredicted degradations	False Positives
Fight	20	43	39	0	4
FPS	27	49	43	0	6
Racing	88	121	113	0	8
RPG	48	87	74	0	13
TOTAL	183	300	269	0	31
%		100%	89.7%	0%	10.3%

Table 5 Results of the effective loss cancelation

Title	Launched games	Predictions	Effective loss cancellations	Network conditions	Latency
CH	18	45	41	A-B-C-D-E-F	low
CR	15	42	28	A-B-C-D-E-F	low
EXT	18	35	33	A-B-C-D-E-F	low
GRA	18	27	23	A-B-C-D-E-F	low
GRI	18	32	31	A-B-C-D-E-F	low
KOF	18	43	34	A-B-C-D-E-F	low
LOTF	18	27	23	A-B-C-D-E-F	low
MXGP	18	33	29	A-B-C-D-E-F	low
TAN	17	47	39	A-B-C-D-E-F	low
CH	4	10	9	A-B-C-D	high
EXT	4	9	9	A-B-C-D	high
GRA	4	6	5	A-B-C-D	high
GRI	4	7	7	A-B-C-D	high
LOTF	4	7	7	A-B-C-D	high
MXGP	4	7	6	A-B-C-D	high
TOTAL	182	377	324		

with and without the algorithm in operation. Across all scenarios, participants consistently reported a significant improvement in playability when the algorithm was activated.

In cases where the algorithm responded effectively to loss cancellation, the instant where the video quality drops due to the bit rate decrease, although noticeable, does not affect gameplay and the game can continue normally in low bandwidth network conditions. On the other hand, bit rate increase had no issues when the network conditions were favorable.

8.2 Comparison with other algorithms

The ABRAGame algorithm shows advantages and better performance with respect to the C3G proposed by [27]. The latter is split between the server and the client. The decisions are taken at the server side, which must receive periodically the measurements performed by the client. This can lead to time misalignment between client and server, as detailed in [27]. On

Table 6 Summary results

Genre	Launched Games	Predictions	Effective loss cancellations	Non-effective loss cancellations
Fight	18	43	34	9
FPS	22	55	50	5
Racing	81	154	129	25
RPG	61	125	111	14
TOTAL	182	377	324	53
%		100%	85.9%	14.1%

the other hand, ABRAGame runs only on the client side, so no side channel is needed between client and server, and temporal misalignment issues are avoided. In [27], the performance of C3G was tested with only one game, and reported in a graphic form, showing the number of lost frames as a function of the ratio between the initial and final bandwidth of the channel in stepwise sudden decrease of bandwidth. This relation varies from 1 (no step) to 3 (i.e., an abrupt change from 15 Mbps to 5 Mbps, as the one shown in figure 3.E.). In C3G, when the relation between the initial and final bandwidth is greater than 1.75 (i.e, 15 Mbps to 8.5 Mbps), the C3G algorithm cannot totally eliminate the frame losses. In general, according to the graphs shown in [27], C3G can only avoid frame losses in less than a 45% of the cases, while ABRAGame can make an effective loss cancellation in the 85.9% of the test cases.

ABRAGame also shows enhancements with respect to the algorithms used by commercial CG platforms, whose study was performed by [22]. According to this study, Stadia needs around 80 s to stabilize after the bandwidth is suddenly decreased by a factor of 2. In the same conditions, PS Now needs 20 s to stabilize, but a with huge decrease of the bit rate the first second the constraint is applied, resulting in a stalled video. GeForce Now shows a reaction time between the two others. According to [21], Stadia attempts to recover from a drop in the available link by entering a transient phase that can last over 200 s. During this time, the quality of experience is heavily affected. In the case of ABRAGame, when the frame loss could not be effectively avoided, the average recovery time was measured as 4.5 s in average, much faster than the values reported in [21] and [22].

9 Next steps and future work

Future works will enhance the prediction of visual degradation, studying in depth the causes of the false positive predictions. Other techniques, such us patterns recognition, or even artificial intelligence, can complement the designed algorithm.

The increase of the coding bit rate, when the network issues disappear, can be enhanced in future works. The current stepwise linear approach can be replaced by an exponential one. In this way, as the available bandwidth increases without predicted losses, the bit rate increase can be done faster.

Finally, interactive subjective tests to evaluate the quality of experience, following the [31] standard, are still pending. These values obtained could be compared with the results obtained using the model presented in [18], as well as with those obtained using alternative assessment methods such as [32].

10 Conclusion

CG platforms have emerged as a promising means to play high-quality video games on low-cost devices. However, this also poses several challenges and issues, as these platforms heavily rely on the network's bandwidth connecting the servers to the clients.

In this work, a real-time algorithm called ABRAGame was formulated, tested and implemented, which automatically controls the encoding bit rate of the CG server in order to avoid visual degradations and to maintain the playability, even with varying network conditions, typically present in wireless networks. The designed algorithm has two parts. The first one predicts future visual degradations. The second, adjusts the encoding video bit rate to avoid the predicted degradation. The full algorithm runs on the client side, in real-time, and can

be easily integrated into the CG platform's application. The final ABRAGame algorithm is easy to implement and consumes very low computer resources.

The developed predictive algorithm was tested with 15 games of different genres, over 6 different network conditions and two different latency values, resulting in over 180 played games. It was found that the algorithm successfully predicts video degradation events with 100% accuracy, although it also exhibited a 10.3% false positive rate. The bit rate adjustment function was tested with 9 games of different genres, over 10 different network conditions, also resulting in over 180 played games. Overall, the proposed ABRAGame algorithm successfully mitigates 85.9% of instances involving frame loss and improves gameplay in all cases. The results were compared to other state-of-the-art algorithm, and with the performance of commercial platforms, showing better and enhanced results.

Acknowledgements The authors would like to thank Alex Amaral, Santiago Erramuspe and the ABYA team for their support and contribution to this work.

Data Availability Data sharing not applicable to this article as no datasets were generated or analyzed during the current study.

Declarations

Conflict of Interests The authors declare that they have no conflict of interest.

References

1. Mattioli M (2021) History of video game distribution, In: IEEE consumer electronics magazine, vol 10, no 2, pp 59–63. https://doi.org/10.1109/MCE.2020.3032778
2. Global Games Market Report [Online]. Accessed on Mar 2024. Available: https://newzoo.com/games-market-reports-forecasts
3. Cai W et al (2016) A Survey on Cloud Gaming: Future of Computer Games. In: IEEE access, vol 4, pp 7605–7620. https://doi.org/10.1109/ACCESS.2016.2590500
4. Shrivastava P, Damle M (2022) Investment decision in cloud gaming-based businesses opportunities: an analysis of the cloud gaming industry, In: 2022 International Conference on Decision Aid Sciences and Applications (DASA), Chiangrai, Thailand, pp 1224-1228. https://doi.org/10.1109/DASA54658.2022.9765298
5. Bukhari SMAH, Baccour E, Bilal K, Shuja J, Erbad A, Bilal M (2023) To transcode or not? A machine learning based edge video caching and transcoding strategy. Comput Electr Eng 109:108741
6. Huang C-Y, Hsu C-H, Chang Y-C, Chen K-T (2013) GamingAnywhere: The first open source cloud gaming system. A Proceedings of the 4th ACM Multimedia Systems Conference, ser. MMSys '13. New York, NY, USA: Association for Computing Machinery
7. Metzger F, Geißler S, Grigorjew A, Loh F, Moldovan C, Seufert M, Hoßfeld T (2022) An introduction to online video game qos and qoe influencing factors. IEEE Commun Surveys Tutorials 24(3):1894–1925. https://doi.org/10.1109/COMST.2022.3177251
8. Jaya I, Li Y, Cai W (2024) Deep reinforcement learning based resource allocation in edge-cloud gaming. Multimed Tool Appl. https://doi.org/10.1007/s11042-024-18337-2
9. Amaral A, Armendariz A, Erramuspe S, Joskowicz J, Liu M (2023) Prediction of video quality degradation on a cloud gaming platform. In: 2023 IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB). https://doi.org/10.1109/BMSB58369.2023.10211223
10. ABYA. [Online]. Accessed on Mar 2024. Available: https://abya.com/es
11. GeForce NOW. [Online]. Accessed on Mar 2024. Available: https://www.nvidia.com/geforce-now
12. Xbox Cloud Gaming (Beta). [Online]. Accessed on Mar 2024. Available: https://www.xbox.com/en-US/cloud-gaming
13. Best Cloud Gaming Services: Xbox Game Pass, GeForce Now and More. [Online]. Accessed on Mar 2024. Available: https://www.cnet.com/tech/gaming/best-cloud-gaming-services/

14. Cloud Gaming to Launch in Latin America with GeForce NOW Powered by ABYA [Online]. Accessed on Aug 2023. Available: https://abya.com/en/news/post/cloud-gaming-launch-latin-america-geforce-now-powered-abya
15. Shea R, Liu J, Ngai EC-H, Cui Y (2013) Cloud gaming: architecture and performance. IEEE Netw 27(4):16–21
16. Sabet SS, Schmidt S, Zadtootaghaj S, Griwodz C, Moller S (2020) Towards the impact of gamers strategy and user inputs on the delay sensitivity of cloud games. In: Twelfth International Conference on Quality of Multimedia Experience (QoMEX) vol 2020, pp 1–3
17. ITU-T Recommendation H.264 (2021) Advanced video coding for generic audiovisual services
18. ITU-T Recommendation G.1072 (2020) Opinion model predicting gaming quality of experience for cloud gaming services
19. NVENC Video Encoder API Programming Guide. [Online]. Accessed on Mar 2024. Available: https://docs.nvidia.com/video-technologies/video-codec-sdk/11.1/nvenc-video-encoder-api-prog-guide/index.html
20. Di Domenico A, Perna G, Trevisan M, Vassio L, Giordano D (2021) A network analysis on cloud gaming: Stadia, geforce now and psnow. Network 1(3):247–260
21. Carrascosa M, Bellalta B (2022) Cloud-gaming: Analysis of google stadia traffic. Comput Commun 188:99–116
22. Graff P, Marchal X, Cholez T, Tuffin S, Mathieu B, Festor O (2021) An analysis of cloud gaming platforms behavior under different network constraints. In: 2021 17th International Conference on Network and Service Management (CNSM), pp 551-557
23. Peñaherrera-Pulla OS, Baena C, Fortes S, Baena E, Barco R (2021) Measuring key quality indicators in cloud gaming: Framework and assessment over wireless networks. Sensors 21(4):1387
24. Marchal X, Graff P, Ky JR, Cholez T, Tuffin S, Mathieu B, Festor O (2023) An analysis of cloud gaming platforms behaviour under synthetic network constraints and real cellular networks conditions. J Netw Syst Manage 31(2):39
25. Tolouei P (2023) Analysing the performance of cloud gaming over a low-earth orbit satellite network
26. Carlucci G, De Cicco L, Holmer S, Mascolo S (2017) Congestion control for web real-time communication. IEEE/ACM Trans Netw 25(5):2629–2642
27. Alós A, Morán F, Carballeira P, Berjón D, García N (2019) Congestion control for cloud gaming over udp based on round-trip video latency. IEEE Access 7:78 882-78 897. https://doi.org/10.1109/ACCESS.2019.2923294
28. Gettys J (2011) Bufferbloat: Dark buffers in the internet. IEEE Internet Comput 15(3):96–96
29. Netem. [Online]. Accessed on Mar 2023. Available: https://wiki.linuxfoundation.org/networking/netem
30. ITU-T Recommendation G.1032 (2017) Influence factors on gaming quality of experience
31. ITU-T Recommendation P.809 (2018) Subjective evaluation methods for gaming quality
32. Cha EY, Jalil Piran M, Suh DY (2024) A gaze-based real-time and low complexity no-reference video quality assessment technique for video gaming. Multimed Tools and Applications 83:20889–20908. https://doi.org/10.1007/s11042-023-16184-1

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.